Campus Route Navigation Using Graph Traversals

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 20
#define NAME_LEN 50
// Graph structure
typedef struct {
char name[NAME_LEN];
int adj[MAX];
int adjCount;
} Location;

Location graph[MAX];
int locationCount = 0;
// Find location index by name
int findLocation(char name[]) {
int i;
for (i = 0; i < locationCount; i++) {
if (strcmp(graph[i].name, name) == 0) {
return i;
}
}
return -1;
}
// Add a new location
void addLocation(char name[]) {
```

```
if (findLocation(name) != -1) {
printf("Location already exists!\n");
return;
}
if (locationCount >= MAX) {
printf("Maximum locations reached!\n");
return;
}
strcpy(graph[locationCount].name, name);
graph[locationCount].adjCount = 0;
locationCount++;
}
// Add a road between two locations
void addRoad(char name1[], char name2[]) {
int u = findLocation(name1);
int v = findLocation(name2);
// Check whether both locations exist
if (u == -1 || v == -1) {
printf("One or both locations not found!\n");
return;
}
// Prevent a road from connecting a location to itself
if (u == v) {
printf("A location cannot have a road to itself!\n");
return;
}
// Check whether the road already exists
for (int i = 0; i < graph[u].adjCount; i++) {
if (graph[u].adj[i] == v) {
```

```
return;
}
}
// Check whether both adjacency arrays have space
if (graph[u].adjCount >= MAX || graph[v].adjCount >= MAX) {
printf("Cannot add road: adjacency list is full!\n");
return;
}
// Add the road in both directions (undirected graph)
graph[u].adj[graph[u].adjCount] = v;
graph[u].adjCount++;
graph[v].adj[graph[v].adjCount] = u;
graph[v].adjCount++;
}
// Display complete campus graph
void displayGraph() {
int i, j;
printf("\n========== ANITS CAMPUS MAP ==========\n");
for (i = 0; i < locationCount; i++) {
printf("%s -> ", graph[i].name);
for (j = 0; j < graph[i].adjCount; j++) {
printf("%s",
graph[graph[i].adj[j]].name);
if (j < graph[i].adjCount - 1)
printf(", ");
}
printf("\n");
}
}
```

```
// BFS - finds shortest path
// based on number of roads
void BFS(int start, int destination) {
int queue[MAX];
int front = 0;
int rear = 0;
int visited[MAX] = {0};
int parent[MAX];
int i;
// Initialize parent array
for (i = 0; i < MAX; i++) {
parent[i] = -1;
}
// Add starting location
queue[rear++] = start;
visited[start] = 1;
// BFS
while (front < rear) {
int current = queue[front++];
// Destination found
if (current == destination)
break;
for (i = 0;
i < graph[current].adjCount;
i++) {
int next = graph[current].adj[i];
if (!visited[next]) {
visited[next] = 1;
parent[next] = current;
```

```
queue[rear++] = next;
}
}
}
// Destination not reachable
if (!visited[destination]) {
printf("\nNo route exists between %s and %s.\n",
graph[start].name,
graph[destination].name);
return;
}
// Reconstruct path
int path[MAX];
int pathCount = 0;
int current = destination;
while (current != -1) {
path[pathCount++] = current;
current = parent[current];
}
// Display route
printf("\n========== SHORTEST ROUTE ==========\n");
printf("\n");
for (i = pathCount - 1; i >= 0; i--) {
printf("%s",
graph[path[i]].name);
if (i != 0)
printf(" -> ");
}
printf("\n");
```

```
printf("\nNumber of roads: %d\n",
pathCount - 1);
printf("=====================================\n");
}
// DFS function
void DFS(int current, int visited[]) {
int i;
visited[current] = 1;
printf("%s ", graph[current].name);
for (i = 0;
i < graph[current].adjCount;
i++) {
int next = graph[current].adj[i];
if (!visited[next]) {
DFS(next, visited);
}
}
}
// DFS traversal
void DFSTraversal(int start) {
int visited[MAX] = {0};
printf("\n========== DFS CAMPUS EXPLORATION ==========\n");
DFS(start, visited);
printf("\n");
}
// Display all locations
void displayLocations() {
int i;
printf("\n========== ANITS LOCATIONS ==========\n");
```

```
for (i = 0; i < locationCount; i++) {
printf("%d. %s\n",
i + 1,
graph[i].name);
}
printf("=====================================\n");
}
// Main function
int main() {
int choice;
char name1[NAME_LEN];
char name2[NAME_LEN];
// ADD ANITS CAMPUS LOCATIONS
addLocation("Main Gate");
addLocation("Open Auditorium");
addLocation("Canteen");
addLocation("Admin Block");
addLocation("CSE");
addLocation("ECE");
addLocation("EEE");
addLocation("Chemical");
addLocation("Mechanical");
addLocation("Food Engine");
addLocation("Bus Area");
addLocation("Hostels");
addLocation("Back Gate");
// ADD ROADS ACCORDING TO YOUR CAMPUS MAP
// Main Gate connections
addRoad("Main Gate", "Open Auditorium");
```

```
addRoad("Main Gate", "Canteen");
addRoad("Main Gate", "Admin Block");
// Admin Block connections
addRoad("Admin Block", "CSE");
addRoad("Admin Block", "ECE");
addRoad("Admin Block", "EEE");
addRoad("Admin Block", "Chemical");
addRoad("Admin Block", "Mechanical");
// CSE side
addRoad("CSE", "ECE");
addRoad("CSE", "EEE");
addRoad("CSE", "Chemical");
addRoad("CSE", "Mechanical");
// ECE side
addRoad("ECE", "CSE");
addRoad("ECE", "EEE");
addRoad("ECE", "Chemical");
addRoad("ECE", "Mechanical");
// EEE side
addRoad("EEE", "CSE");
addRoad("EEE", "ECE");
addRoad("EEE", "Chemical");
addRoad("EEE", "Mechanical");
// Chemical side
addRoad("Chemical", "Mechanical");
addRoad("Chemical", "EEE");
addRoad("Chemical", "ECE");
addRoad("Chemical", "CSE");
// Mechanical side
```

```
addRoad("Mechanical", "Food Engine");
addRoad("Mechanical", "Chemical");
addRoad("Mechanical", "ECE");
addRoad("Mechanical", "EEE");
addRoad("Mechanical", "CSE");
addRoad("Mechanical", "Bus Area");
// Food Engine side
addRoad("Food Engine", "Bus Area");
// Bus Area connections
addRoad("Bus Area", "Hostels");
addRoad("Bus Area", "Back Gate");

while (1) {
printf("\n\n");
printf("================================================\n");
printf("       ANITS CAMPUS ROUTE NAVIGATION\n");
printf("================================================\n");
printf("1. Display Locations\n");
printf("2. Display Campus Map\n");
printf("3. Find Shortest Route (BFS)\n");
printf("4. Explore Campus (DFS)\n");
printf("5. Exit\n");
printf("================================================\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
// DISPLAY LOCATIONS
case 1:displayLocations();
break;
```

```
// DISPLAY CAMPUS MAP
case 2:displayGraph();
break;
// FIND SHORTEST ROUTE
case 3:displayLocations();
printf("\nEnter starting location: ");
scanf(" %[^\n]", name1);
printf("Enter destination: ");
scanf(" %[^\n]", name2);
{
int start =
findLocation(name1);
int destination =
findLocation(name2);
if (start == -1 ||
destination == -1) {
printf("\nInvalid location name!\n");
}
else {
BFS(start, destination);
}
}
break;
// DFS CAMPUS EXPLORATION
case 4:displayLocations();
printf("\nEnter starting location for DFS: ");
scanf(" %[^\n]", name1);
{
int start =
```

```
findLocation(name1);
if (start == -1) {
printf("\nInvalid location name!\n");
}
else {
DFSTraversal(start);
}
}
break;
// EXIT
case 5:printf("\n");
printf("Thank you for using\n");
printf("ANITS Campus Route Navigation!\n");
exit(0);
// INVALID CHOICE
default:printf("\nInvalid choice!\n");
printf("Please try again.\n");
}
}
return 0;
}
```